

OS Project 2 - Project Report

Distributed Prime Counter using gRPC

A. Design and Implementation

a. Architecture Overview

The system consists of four distinct processes communicating exclusively via gRPC over TCP:

- main process: hosts the Dispatcher (port 5001) and Consolidator (port 5002) as goroutines. Divides the data file into N-byte segments, pre-fills an internal buffered job queue, starts both gRPC servers, then blocks until all results are received.
- fileserver process: opens the data file once at startup and listens on port 5003. Serves FetchSegment requests using server-side streaming gRPC, sending the segment's bytes in C-byte chunks using file.ReadAt() for concurrent-safe access without a mutex.
- worker processes (M total, started manually): each worker makes a unary PullJob RPC to the dispatcher, a server-side streaming FetchSegment RPC to the fileserver, counts primes in each received chunk, then makes a unary PushResult RPC to the consolidator. Workers loop until PullJob returns Done=true.
- primes_config.txt: a plain ASCII file storing the address and port of each server, read at startup by all processes. Allows processes to run on different machines without recompiling.

b. gRPC Services (defined in primes.proto)

Three services are defined:

- Dispatcher.PullJob (unary): worker pulls one job from the internal queue. Returns Done=true when the queue is closed.
- FileServer.FetchSegment (server-side streaming): worker requests a segment by byte offset and size; fileserver streams it back in C-byte chunks using file.ReadAt() for concurrent safety.
- Consolidator.PushResult (unary): worker pushes its segment prime count and job count. Consolidator accumulates totals and closes the job queue when all expected results are received, signalling workers to exit.

c. Job Queue and Termination

The dispatcher's internal job queue is a buffered Go channel pre-filled with all segment descriptors before any worker connects. When the consolidator receives all expected results, it closes the job queue channel. Subsequent PullJob calls then return Done=true, causing all workers to exit their loop cleanly. The main process unblocks via a done channel and prints the final results.

d. Concurrency and Correctness

- No race conditions: the consolidator uses a sync.Mutex to protect shared state (totalPrimes, received count, workerJobs map) since multiple gRPC handler goroutines may call PushResult concurrently.
- Maximum concurrency: workers operate fully independently with no synchronization between them. The dispatcher serves PullJob calls concurrently via gRPC's built-in goroutine-per-request model.
- file.ReadAt() is used in the fileserver instead of Seek+Read, allowing concurrent segment reads from multiple workers without a mutex.
- Workers use a pull-based job model ensuring self-scheduling with minimal idle time and excellent load balance across all M workers.

e. Minimum Information Communicated

- PullJob response: 1 bool + 3 int64 = 25 bytes per job
- FetchSegment: raw file bytes only, no per-chunk metadata beyond gRPC framing
- PushResult request: 2 int64 + 2 int32 = 24 bytes per result
- Workers never send raw file data back — only prime counts

f. Prime Detection

Numbers are decoded from raw bytes as little-endian uint64 (8 bytes each), identical to Project 1. Primality is tested using Go's math/big.ProbablyPrime(20), a Miller-Rabin probabilistic test with 20 rounds, giving negligible false-positive probability.

B. Q1 - Least Expected Elapsed Time for a 1GB File

The least elapsed time for a random 1GB datafile was achieved with M=16 workers, N=65536 (64KB segments), C=4096 (4KB chunks).

Command:

```
go run fileserver/fileserver.go data1gb.dat primes_config.txt  
./run.sh 65536 4096 16 data1gb.dat primes_config.txt
```

Parameter	Value
M (workers)	16
N (segment size)	65536 bytes (64KB)
C (chunk size)	4096 bytes (4KB)
File size	1 GB
Total primes found	3,101,315
Elapsed time	80,674 ms (~1.34 min)
Min jobs per worker	1,014
Max jobs per worker	1,050
Avg jobs per worker	1,024.0
Median jobs per worker	1,020.5

With M=16 workers, the 1GB file (16,384 segments of 64KB) is processed in roughly equal shares of ~1024 jobs per worker. The tight spread between min (1,014) and max (1,050) demonstrates excellent load balancing via the pull-based job queue.

C. Q2 - Largest File Processed Within 3 Minutes

Command:

```
./run.sh 65536 4096 16 data2gb.dat primes_config.txt
```

Parameter	Value
M (workers)	16
N (segment size)	65536 bytes (64KB)
C (chunk size)	4096 bytes (4KB)
File size	2 GB
Total primes found	6,194,441
Elapsed time	176,187 ms
Min jobs per worker	2,003
Max jobs per worker	2,076
Avg jobs per worker	2,048.0
Median jobs per worker	2,048.5

Generate the test files with:

```
head -c 2G /dev/urandom > data2gb.dat
```

```
head -c 4G /dev/urandom > data4gb.dat
```

D. Q3 - Elapsed Time as a Function of M

Using the largest file from Q2 with N=65536, C=4096, elapsed time was measured for M = 4, 8, and 16 workers.

M (workers)	Elapsed (ms)	Min Jobs	Max Jobs	Avg Jobs	Median Jobs
4	151,218	4,075	4,117	4,096.0	4,096.0
8	94,180	2,038	2,060	2,048.0	2,047.5
16	80,674	1,014	1,050	1,024.0	1,020.5

As M increases from 4 to 16, elapsed time decreases significantly — roughly halving from M=4 to M=8. The gains diminish from M=8 to M=16 as the number of workers begins to approach the number of physical CPU cores on the machine. Load balancing is excellent across all values of M — the pull-based job queue ensures workers self-schedule with minimal idle time, as seen in the tight min/max job count spread for every test.

E. Key Differences from Project 1

Project 1	Project 2
Go channels for communication	gRPC over TCP
Workers are goroutines	Workers are separate OS processes
Shared memory (file access)	No shared memory — fileserver streams data
Single process	4+ separate processes
No network overhead	gRPC framing per chunk and RPC call
wg.WaitGroup for termination	Close job queue channel via consolidator
jobCounts[]int shared slice	workerJobs map[int32]int32 with mutex